Architecture 1001:
x86-64 Assembly
The Stack

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

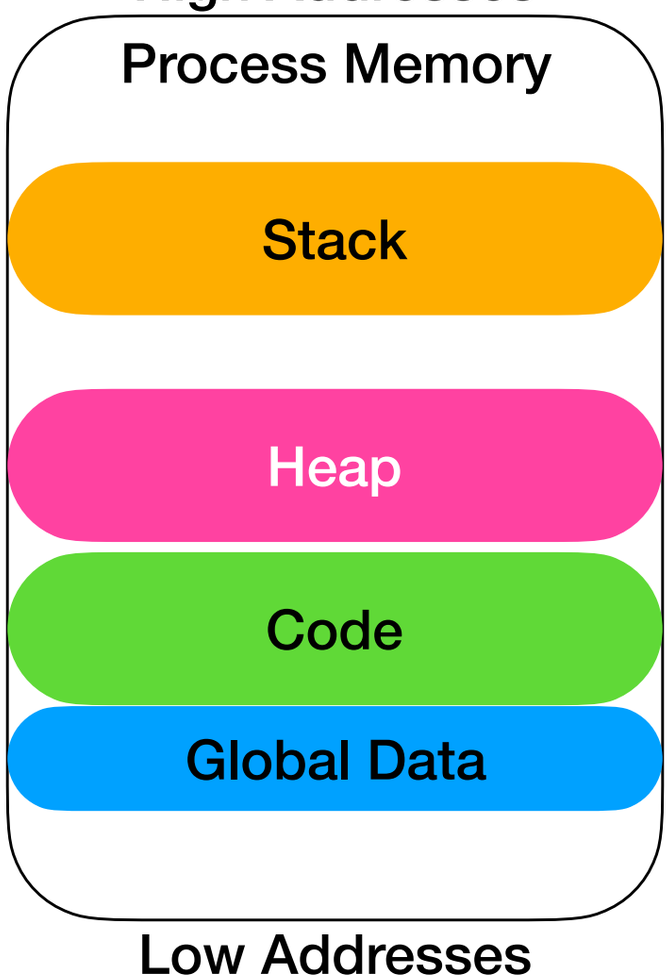
The Stack

- A stack is a Last-In-First-Out (LIFO) data structure where data is "pushed" on to the top of the stack and "popped" off the top



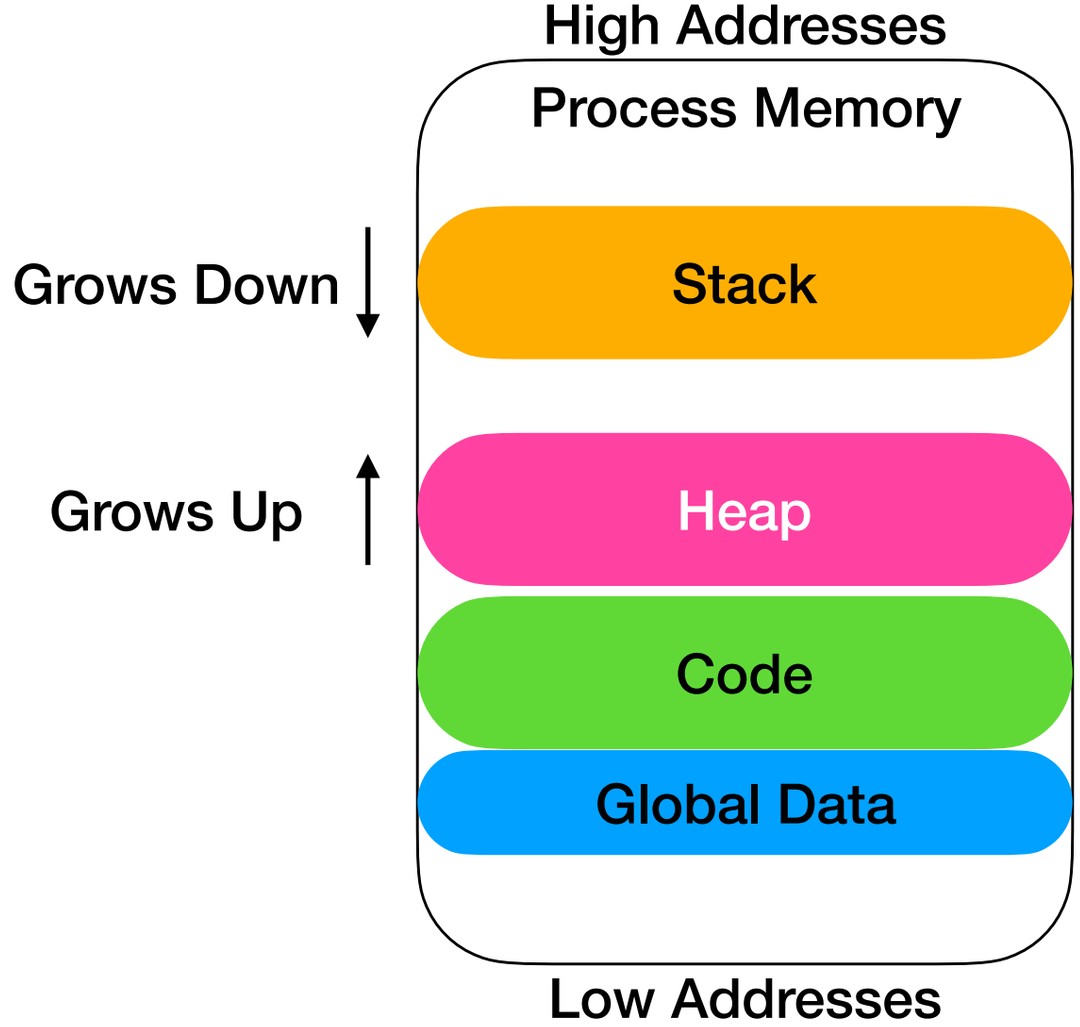
- The stack is a conceptual area of main memory (RAM) which is designated by the OS when a program is started
  - Different OSes start it at different addresses by their own convention, or if they're using address space layout randomization (ASLR)

The Stack 2

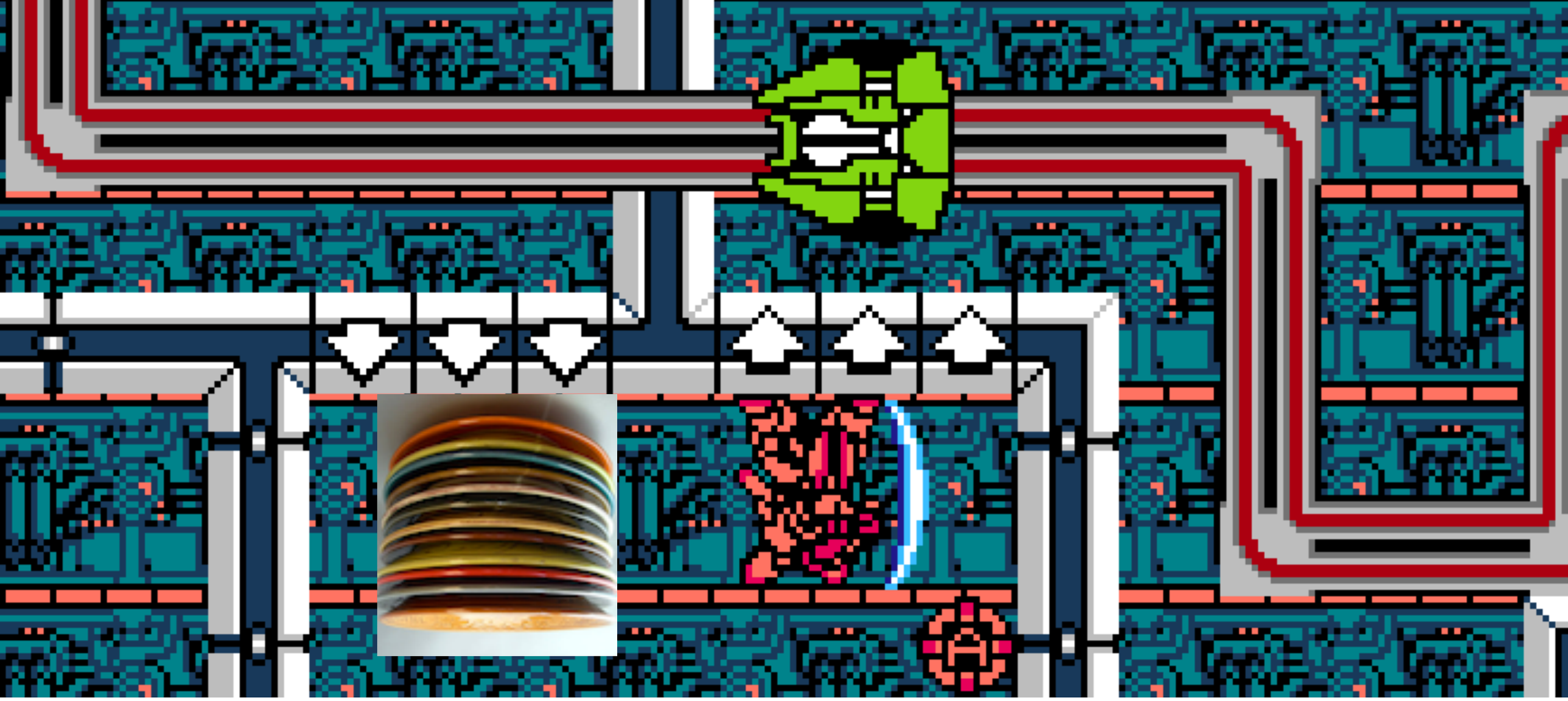

High Addresses
Process Memory
Stack
Heap
Code
Global Data
Low Addresses

The Stack 2

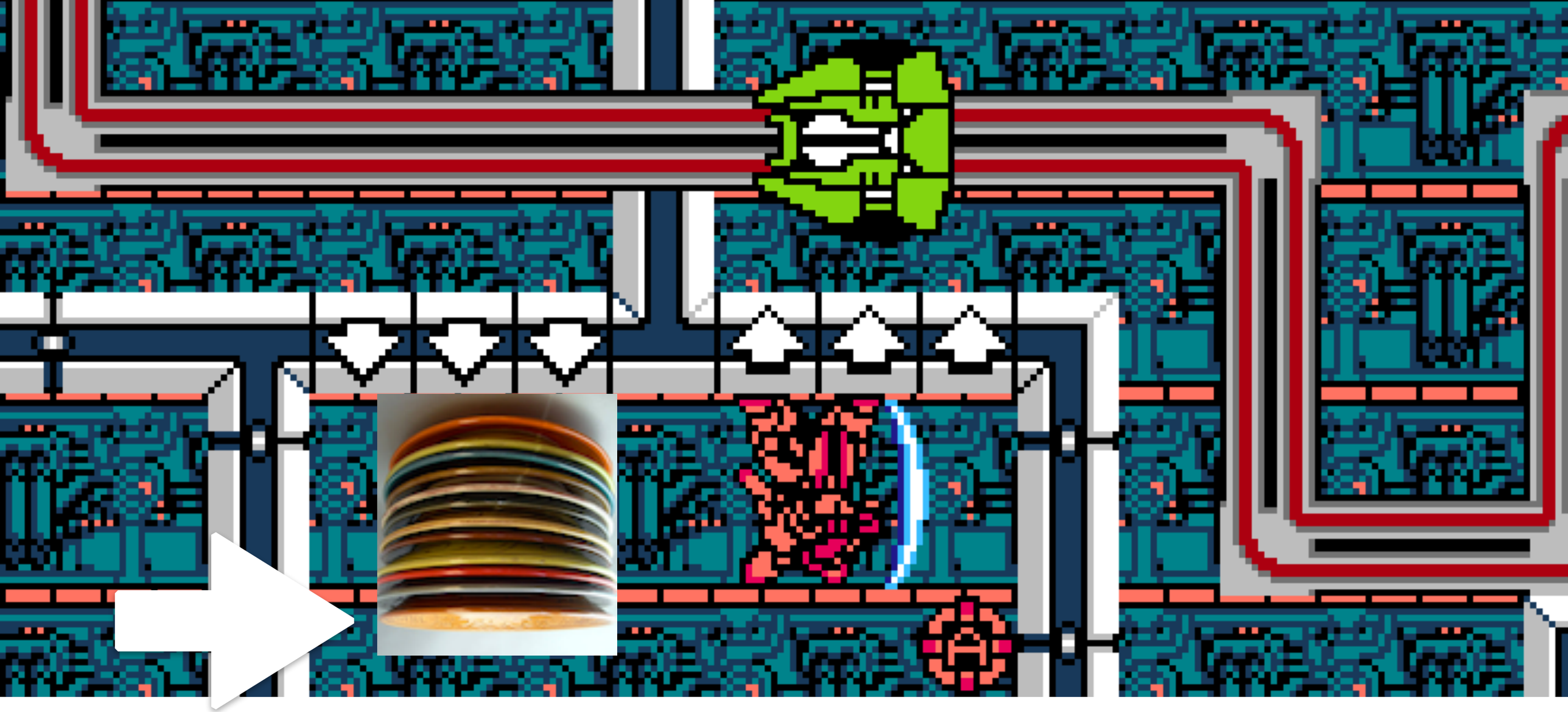
- By convention **the stack *grows* toward lower memory addresses**. Adding something to the stack means the top of the stack is now at a lower memory address



Book page 107



This message brought to you by M-308 Gunner from MetalStorm!



This message brought to you by M-308 Gunner from MetalStorm!



This message brought to you by M-308 Gunner from MetalStorm!



The enemy's *plate* is *down*!



Everything you know is wrong! Black is white, up is down, and short is long!

The Stack 3

- RSP points to the top of the stack - *the lowest address which is being used*
  - While data *will* exist at addresses beyond the top of the stack, *it is considered undefined*

The Stack 4

The Stack 4

- What can you find on the stack?

The Stack 4

- What can you find on the stack?
  - "Return addresses" so a called function can return back to the function that called it

The Stack 4

- What can you find on the stack?
  - "Return addresses" so a called function can return back to the function that called it
  - Local variables

The Stack 4

- What can you find on the stack?
  - "Return addresses" so a called function can return back to the function that called it
  - Local variables
  - S*ometimes* used to pass arguments between functions

Book page 107

The Stack 4

- What can you find on the stack?
  - "Return addresses" so a called function can return back to the function that called it
  - Local variables
  - S*ometimes* used to pass arguments between functions
  - Save space for registers so functions can share registers without smashing the value for each other

The Stack 4

- What can you find on the stack?
  - "Return addresses" so a called function can return back to the function that called it
  - Local variables
  - S*ometimes* used to pass arguments between functions
  - Save space for registers so functions can share registers without smashing the value for each other
  - Save space for registers when the compiler has to juggle too many in a function

The Stack 4

- What can you find on the stack?
  - "Return addresses" so a called function can return back to the function that called it
  - Local variables
  - S*ometimes* used to pass arguments between functions
  - Save space for registers so functions can share registers without smashing the value for each other
  - Save space for registers when the compiler has to juggle too many in a function
  - Dynamically allocated memory via alloca()

Simple Stack Diagram

stack bottom

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```

stack top

Simple Stack Diagram

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```

stack bottom



main()

frame

stack top

Book page 113

Simple Stack Diagram

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```



Simple Stack Diagram

```c
#include <stdio.h>

int bar(int y){
    int a = 3*y;
    printf("bar returned %d", a);
    return a;
}

int foo(int x){
    int b = 5*x;
    printf("foo passed %d", b);
    return bar(b);
}

int main(){
    int c = foo(7);
    printf("main passed %d", c);
}
```

